Week 12

Nav2, and Merge 2

Jaerock Kwon, PhD
Associate Professor
Electrical and Computer Engineering
University of Michigan-Dearborn

★ Merge 2

Simulation + Software

The claim

> *The twin runs **measured** geometry, and the functional test suite is green in CI.*

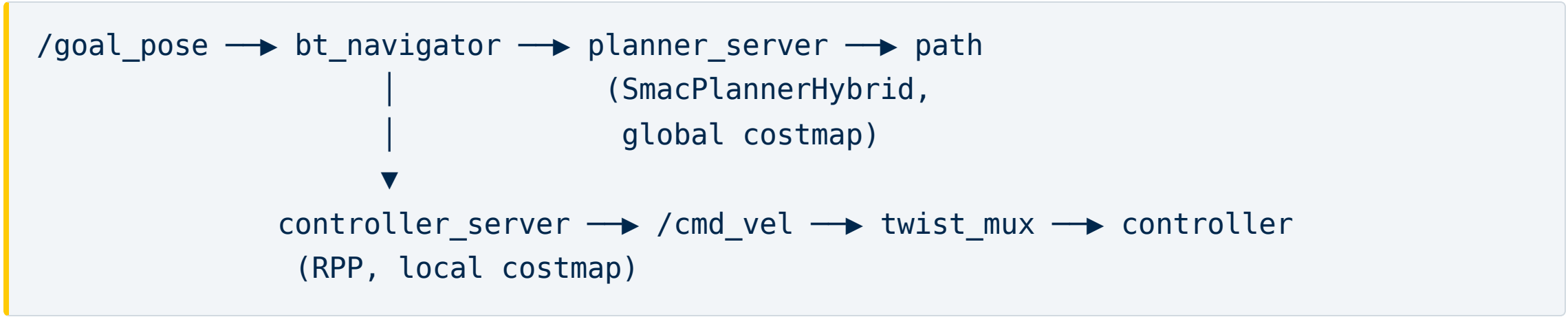
Measured geometry in the twin	No `TODO measure` left in the values that matter
Both turning radii re-derived	Planner **and** controller, together, from measured L and δ_max
Sim vs real, quantified	Steering range, rate limit, wheelbase, latency — **within 10%**, or named
Functional tests	`colcon test` asserts something that is not a linter
CI	ROS build and tests run in Actions, which today builds only docs

This is where the twin stops being a demo and becomes a test rig.

Until today it ran on numbers derived from a vendor's marketing dimensions.

The Nav2 pipeline

Goal to velocity



Plus `behavior_server` for recoveries, and `lifecycle_manager` bringing every node up in order.

Costmaps

	Global	Local
Covers	The whole known map	A rolling window
Built from	Static map + obstacles	Live sensor data
Used by	The planner	The controller
Frame	`map`	`odom`

Layers stack: `StaticLayer` · `ObstacleLayer` · `InflationLayer`

MRider uses a footprint, not a radius

```
footprint: [[0.81, 0.28], [0.81, -0.28],
            [-0.18, -0.28], [-0.18, 0.28]]
```

0.81 m forward, 0.18 m back. Not a mistake.

`base_link` is at the **rear axle**, so most of the car is in front of it.

A circular approximation would be badly wrong for this shape.

Why the planner was replaced

NavFn plans for a free-turning point

For a differential robot, nearly true. For a car, not.

> *NavFn routinely returned paths this car cannot drive, and RPP rejected them mid-follow with* `detected collision ahead!` *on geometry that was never feasible.*

No controller can follow an infeasible path.

Tuning could not have fixed this. The fault was upstream.

Dropped on geometry, without a trial run

Both **NavFn** and **DWB** were rejected before being tried.

Unusual, and deliberate: when the reason is **kinematic** rather than empirical, running the experiment to confirm it burns a week to learn what the geometry already said.

SmacPlannerHybrid

```
plugin: "nav2_smac_planner/SmacPlannerHybrid"
motion_model_for_search: "REEDS_SHEPP"
minimum_turning_radius: 1.6
reverse_penalty: 2.5
angle_quantization_bins: 72
```

Searches **(x, y, heading)**, not (x, y).

It **cannot** return an undrivable path — undrivable paths are not in its search space.

Reeds-Shepp curves include reversing segments. Dubins curves do not.

The two changes only work together

`allow_reversing: true` was **inert** before the planner swap.

RPP reverses only where the **path** reverses,
and NavFn never produced such a path.

A parameter that is on but has no effect looks exactly like one that works.

Verify capabilities by exercising them, not by reading configuration.

The verified result

A goal **2.5 m directly behind** the vehicle —
geometrically impossible forward-only at a 1.6 m radius.

The vehicle **reversed 2.1 m in a straight line**, heading unchanged at ~3°, arriving 0.40 m from the goal.

Before the swap, that manoeuvre had **no solution at all.**

`reverse_penalty: 2.5` keeps it a last resort — because reverse is **blind.** No rear sensing. Free in simulation; not free on the vehicle.

Behavior trees

One change from the stock tree

```
<RoundRobin name="RecoveryActions">
  <Sequence name="ClearingActions">
    <ClearEntireCostmap ... local .../>
    <ClearEntireCostmap ... global .../>
  </Sequence>
  <Wait wait_duration="5"/>
</RoundRobin>
```

`Spin` and `BackUp` removed — for **two different reasons.**

Spin vs BackUp

`Spin` — the vehicle has **no mechanism** for rotating in place.
It simply cannot.

`BackUp` — a **blind** reverse reflex, on a vehicle with no rear sensing.

Note this is **not** "the car cannot reverse." It can, and does.
The planner reverses along a path computed **against a costmap**.
`BackUp` just reverses.

Only the first is defensible here.

F-N2: the symptom points at the wrong thing

`behavior_server` loads only `wait`. A **stock** tree against it fails:

```
Exception when loading BT: Action server spin not available
```

`bt_navigator` stays `unconfigured`, `lifecycle_manager` aborts bringup of the **whole** stack. And:

> *planner, controller and both costmaps report* ***`active`*** *while navigation is simply absent and no goal is ever accepted.*

Four of five components report healthy.

Both trees must be overridden

`bt_navigator` loads `navigate_to_pose` **and** `navigate_through_poses` at configure time.

Fix only the one you use, and bringup still fails — identically, and just as misleadingly.

F-N4: when recovery runs out

Given a goal 0.05 m from a wall, RPP correctly drove to it and refused the last metre.

That left the car nosed up to the obstacle, where every subsequent plan was infeasible — and with no `Spin` and no blind `BackUp`, **there is no autonomous way out.**

Recovery on this platform ends with a human taking the RC sticks.

Not a gap in the software. The last layer of the authority ladder is a person, by design.

Track work

Track	
Electrical	Stage 4 prep — full integration on the chassis, wheels on stands
Chassis	Support Stage 4; no logic-rail brownout under steering stall
Simulation	Hallway world matching the real space; the ≥ 30 m loop-closure gate
Software	**`mitt_hardware` against the real Teensy**

Merge 3 next week is the ADR-SW4 gate:
the identical teleop launch runs in sim and on hardware,
differing only in the plugin.

This is the claim the entire twin exists to support.